資訊安全導論 HW5 四資工三甲 B10632032 黃千綾

分工	
	Key generation
四資工三甲 B10632032 黃千綾	Signing and verification

建置環境

作業系統：Win 10程式語言：Python 3

操作方式

1. Key generation：python ./DSA.py -keygen {key_length}
2. Signing：python ./DSA.py -sign
3. Verification：python ./DSA.py -verify

執行結果

```
(base) D:\Aya\台科\三上\資訊安全導論\HW\Information_Security_Class\hw5>python ./DSA.py -keygen 160

(base) D:\Aya\台科\三上\資訊安全導論\HW\Information_Security_Class\hw5>python ./DSA.py -sign message

(base) D:\Aya\台科\三上\資訊安全導論\HW\Information_Security_Class\hw5>python ./DSA.py -verify message
valid
```

程式碼解說

- main()：讀取指令並執行

```python
def main():
    if(len(sys.argv) == 3):
        # ./DSA.py -keygen {key length}
        if(sys.argv[1] == '-keygen'):
            generate_key(int(sys.argv[2]))
        # ./DSA.py -sign {message}
        elif(sys.argv[1] == '-sign'):
            sign(sys.argv[2])
        # ./DSA.py -verify {message}
        elif(sys.argv[1] == '-verify'):
            verify(sys.argv[2])
# end main()
```

- generate_key(key_length)：產生p, q, alpha, beta, d，並寫檔

```python
def generate_key(key_length):
    q = generate_prime(key_length)
    p = q << (1024-160)
    while (not (is_prime(p + 1)) and (len(bin(p)[2:]) == 1024)):
        p = p + q
    p = p + 1

    while ((len(bin(p)[2:]) != 1024)):
        q = generate_prime(key_length)
        p = q << (1024-160)
        while (not (is_prime(p + 1)) and (len(bin(p)[2:]) == 1024)):
            p = p + q
        p = p + 1
```

```
    alpha = None
    for h in range(2, p-1):
        # a = h^((p-1)//q) % p
        alpha = square_and_multiply(h, (p - 1) // q, p)
        if(square_and_multiply(alpha,q,p) == 1):
            break;

    d = random.randrange(1, q)
    beta = square_and_multiply(alpha, d, p)

    fp = open("public_key.txt", "w")
    fp.write(str(p) + "\n")
    fp.write(str(q) + "\n")
    fp.write(str(alpha) + "\n")
    fp.write(str(beta) + "\n")
    fp.close()

    fp = open("private_key.txt", "w")
    fp.write(str(d) + "\n")
    fp.close()
# end generate_key()
```

- sign(message)：讀檔、產生r和s並寫檔

```
def sign(message):
    # 讀公鑰
    file = open('public_key.txt','r')
    p = int(file.readline())
    q = int(file.readline())
    a = int(file.readline())
    b = int(file.readline())
    file.close()
    # 讀私鑰
    file = open('private_key.txt','r')
    d = int(file.readline())
    file.close()

    # 0 < k < q
    k = random.randrange(1,q)
    # r = (a^k % p) % q
    r = square_and_multiply(a, k, p) % q
    # s = (SHA(x) + d * r) * k^-1 % q
    sha_x = int(hashlib.sha1(message.encode('utf8')).hexdigest(),16)
    s = (( sha_x + d * r) * find_mode_inverse(k,q)) % q
    # 存簽章
    file = open('signature.txt','w')
    file.write( str(r) + '\n' + str(s))
    file.close()
# end sign()
```

- verify(message)：讀檔、產生v並印出驗證結果

```python
def verify(message):
    # 讀公鑰
    file = open('public_key.txt','r')
    p = int(file.readline())
    q = int(file.readline())
    a = int(file.readline())
    b = int(file.readline())
    file.close()
    # 讀簽章
    file = open('signature.txt','r')
    r = int(file.readline())
    s = int(file.readline())
    file.close()

    # w = s^-1 % q
    w = find_mode_inverse(s,q) % q
    # u1 = w * SHA(x) % q
    sha_x = int(hashlib.sha1(message.encode('utf8')).hexdigest(),16)
    u1 = w * sha_x % q
    # u2 = w * r % q
    u2 = w * r % q
    # v = (a^u1 * b^u2 % p) % q
    v = (square_and_multiply(a,u1,p)*square_and_multiply(b,u2,p)) % q
    if(v == r):
        print('valid')
    else:
        print('invalid')
# end verify(message)
```

- generate_prime(key_length)：依據指定長度產生大質數

```python
def generate_prime(key_length):
    while True:
        # generate random bits
        number = random.getrandbits(int(key_length))
        # apply a mask to set MSB and LSB to 1
        number |= (1 << int(key_length) - 1) | 1
        if is_prime(number):
            return number
```

- find_mode_inverse(a, m)：以 Euclid’ s Extended Algorithm 找出 a 的模反元素

```python
def find_mode_inverse(a, m):
    if math.gcd(a, m) != 1:
        return None
    u1, u2, u3 = 1, 0, a
    v1, v2, v3 = 0, 1, m

    while v3 != 0:
        q = u3 // v3
        v1, v2, v3, u1, u2, u3 = (
            u1 - q * v1), (u2 - q * v2), (u3 - q * v3), v1, v2, v3
    return u1 % m
```

- is_prime(number)：判斷是否為質數（小質數使用一般判斷方式，大質數用rabin_miller）

資訊安全導論 HW5 四資工三甲 B10632032 黃千綾

```python
def is_prime(number):    # 判斷質數

    if(number < 2):
        return False
    elif(number == 2):
        return True
    elif(number % 2 == 0):
        return False
    # 測試 2 - sqrt(n) 中有無 n 的因數
    elif(number < 1000):
        temp = math.ceil(math.sqrt(number))
        for i in range(3, temp, 2):
            if(number % i == 0):
                return False
        return True
    else:
        return rabin_miller(number)
# end is_prime()
```

- rabin_miller(number)：測試是否為質數

```python
def rabin_miller(number):
    # 找 u 和 r 使 p' -1 = 2**u * r
    r = number - 1
    u = 0
    while r % 2 == 0:
        r = r // 2
        u += 1
    # 做五次測試
    for test in range(5):
        # 找 a 使 a**r % p' != 1
        a = random.randrange(2, number - 1)
        temp = square_and_multiply(a, r, number)
        if temp != 1:
            # 找 a 使 a**r**2j != (p'-1) % p'
            j = 0
            while temp != (number - 1):
                if j == u - 1:
                    # 找到的話為合數
                    return False
                else:
                    j = j + 1
                    temp = square_and_multiply(temp, 2, number)
    # 找不到就有可能是質數
    return True
# end rabin_miller(num)
```

- square_and_multiply(x, h, n)：加速計算 x**h%n

資訊安全導論 HW5 四資工三甲 B10632032 黃千綾

```python
def square_and_multiply(x, h, n):    # Output x**h % n
    y = x
    bitH = [int(x) for x in bin(h)[2:]]
    for i in range(1, len(bitH)):
        y = y**2 % n
        if bitH[i] == 1:
            y = y * x % n
    return y
# end square_and_multiply
```

遇到困難與心得

繼上次RSA一直在產生某些數字上遇到困難，這次也一樣，在產生key的部分很碰壁。Sign和verify都是既定的公式，但在產生key的時候，p和q好像一直選不好。只好去和其他同學討論，也有上網搜尋一些相關的演算法，然後試著去理解。但是無論如何，透過產生金鑰達到安全的傳輸的整個流程還是滿有趣的。在應用上可能還是會傾向直接使用套件，至於比較深的數學理論，可能就比較無法太深入。